Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Examples: Using the API to Update Tags, Status, and Comments for Events

This section describes how to use the Stellar Cyber **update_ser** API to perform the following tasks for an event:

- Add or delete custom **Tags**
- Update the **Status** or **Comments** fields

Any changes you make via the API to these fields are visible in the Event Details display for the event.

Refer to **Configuring API Authentication** for general requirements to use the API.

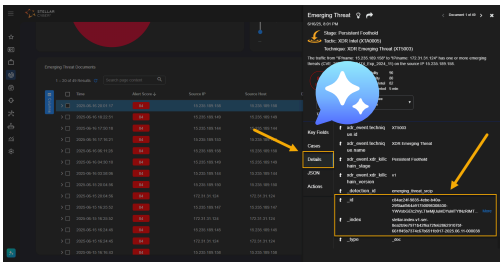
Finding the Index and ID for an Event

To find the `index` and `_id`:

1. Log in to Stellar Cyber.
2. Navigate to the event.
3. Click **More Info** for more information on the event. The Event Details screen appears.
4. Click the **Details** entry in the left navigation panel (or **JSON**) and scroll to the bottom of the display.
5. The `_index` field is the index, and `_id` field is the ID.

The following example illustrates this procedure for an Alert:

1. Navigate to the **Alerts** page and scroll to the Alerts table.
2. Click the **View** button for the Alert whose Index and ID you want to retrieve.
3. Scroll to the bottom of the Alert Detail page where the **Documents** supporting the alert are listed and click the **More Info** button.
4. Select the Details tab in the details panel.
5. Scroll to the bottom of the listed fields and locate the **_id** and **_index** fields, as illustrated in the figure below.



6. Use these fields in your API call.

Using update_ser to Add or Delete Custom Tags for Events

You can use the **update_ser** API to add or delete custom tags for events. As summarized in the table below, your call must have the following:

- The header and the `/connect/api/update_ser` path.
- The index and ID of the event.
- The `event_tag` field.
- The `op` field set to either `add` or `delete`, depending on the type of operation you are performing.
- The tag to be added or deleted in the `tag` field.

Task	Description	Parameters
Add a Tag	/connect/api/update_ser	{ "index": "Index", "_id": "ID", "event_tag": { "op":"add", "tag": "my_tag" } }
Delete a Tag	/connect/api/update_ser	{ "index": "Index", "_id": "ID", "event_tag": { "op":"delete", "tag": "my_tag" } }

Procedure: Using a Python Script to Update the Tag Field for an Event

You can use POST calls to add/delete Tags for an event. You can use any language to make the call. The example below uses Python to add two tags to a specific event. You can copy and paste the example below, replacing the fields in **bold**to match your environment.

1. Find the API key for the account used to make the API call.
2. Find the Event ID and Index for the event whose tag you want to add or delete.
3. Create a Python script similar to the following:

```
headers={'Accept': 'application/json', 'Content-type': 'application/json'}

elastic_url ='https:// YourStellarCyberServer/connect/api/update_ser'

json_data = {

"index": "Index",

"_id": "ID",

"event_tag": {

"op": "add",

"tag": "my_tag"

}

}

query = json.dumps(json_data)

response = requests.post(elastic_url, auth=('Username','API Key'),
data=query, verify=False, headers = headers)

print (response.text)
```

4. Enter your own information for the arguments in **bold** using the table below and run the script.

Argument	Description
Username	User name of the admin making the call
API Key	API key for that user name (edit a user on the **Admin** | **User Management** page to generate an API key)
YourStellarCyberServer	The URL or IP address of your Stellar Cyber server
Index	The index to be queried
ID	The ID of the event to change
event_tag	Must be included to indicate that you are adding or deleting a tag for the specified event.
op	Can be either add or delete, depending on the type of operation you want to perform.
tag	The actual tag to be added or deleted for the specified event.

Examples

The following examples demonstrate common ways to use the **update_ser** API to add and delete tags. These examples use Python, though any programming language will work.

Example: Using a Python Script to Update the Tag Field for an Event

The following example uses a Python script to add a "You_Are_It" tag to an event with the following details:

- **Stellar Cyber DP IP Address** –192.168.12.25
- **Username:API Key** – ohtani:UmT58sxx3We52W5y
- **Index** – aella-ser-2022.03.01-
- **ID** – qe2RR38Boj-otBRJ5J4w

```
#!/usr/bin/env python

import json

import requests

headers={'Accept': 'application/json', 'Content-type': 'application/json'}

headers={'Accept': 'application/json', 'Content-type': 'application/json'}

elastic_url ='https://10.33.6.2/connect/api/update_ser'

json_data = {

"index": "aella-ser-2022.03.01-",

"_id": "qe2RR38Boj-otBRJ5J4w",

"event_tag": {

"op": "add",

"tag": "You_Are_It"

}

}

query = json.dumps(json_data)

response = requests.post(elastic_url, auth=('ohtani', '0MGDNJ5dcHPtNSWy'),
data=query, verify=False, headers = headers)

print (response.text)
```

Values in bold are the values you must supply as part of a similar query.

Example: Using a Python Script to Delete the Tag Field for an Event

Now, we'll use a different script to delete the "You_Are_It" tag from that same event. The only change from the previous script it to set **op** to **delete** instead of **add**.

```
#!/usr/bin/env python

import json

import requests

headers={'Accept': 'application/json', 'Content-type': 'application/json'}

headers={'Accept': 'application/json', 'Content-type': 'application/json'}
```

```
elastic_url ='https://10.33.6.2/connect/api/update_ser'

json_data = {

"index": "aella-ser-2022.03.01-",

"_id": "qe2RR38Boj-otBRJ5J4w",

"event_tag": {

"op": "delete",

"tag": "You_Are_It"

}

}

query = json.dumps(json_data)

response = requests.post(elastic_url, auth=('ohtani', '0MGDNJ5dcHPtNSWy'),
data=query, verify=False, headers = headers)

print (response.text)
```

Using update_ser to Update the Status and Comments Fields for Events

You can use POST calls to update the **Status** and **Comments** fields for an event. You can use any language to make the call. For our example, which is formatted so you can cut and paste, replacing the text in **bold**, we use Python.

 For a POST call you need the index and ID of the event whose fields you want to update.

```
headers={'Accept': 'application/json', 'Content-type': 'application/json'}

elastic_url ='https:// YourStellarCyberServer/connect/api/update_ser'

json_data = {

"index": "Index",

"_id": "ID",

"status": "Status",

"comments": "Comment")

}

query = json.dumps(json_data)

response = requests.post(elastic_url, auth=('Username','API Key'), data=query,
verify=False, headers = headers)

print (response.text)
```

Your call must have the header and the `/connect/api/update_ser` path, and must have either `status` or `comments` (or both). Enter your own information for the arguments in **bold**.

Argument	Description
Username	Username of the admin making the call
API Key	API key for that user name (edit a user on the **Admin** | **User Management** page to generate an API key)
YourStellarCyberServer	The URL or IP address of your Stellar Cyber server
Index	The index to be queried
ID	The ID of the event to change
Status	You can change the status to: • New • In Progress • Ignored • Closed
Comments	You can add whatever comments you want

The contents of response.txt indicate success or failure and will be similar to:

- **OK 200**—your API call was successful
- **Invalid event status value: closed**—your API call failed because the status is invalid (in this case due to the lower case "c")
- **Missing fields to be updated for the event**—your API call failed because you left out the fields to be updated (**Status** or **Comments**)

For an example with the arguments filled in:

```
headers={'Accept': 'application/json', 'Content-type': 'application/json'}

elastic_url ='https://192.168.1.24/connect/api/update_ser'

json_data = {

"index": "aella-ser-1610496070892-",

"_id": "7PW5SXcBlxPU3jFcFJ_s",

"status": "Closed",

"comments": "Analysis complete.",

}

query = json.dumps(json_data)

response = requests.post(elastic_url, auth=('admin','APISquared'), data=query,
verify=False, headers = headers)

print (response.text)
```

This example performs a POST call as the user **admin** with the obviously fake API key of **APISquared**. The call is to the Stellar Cyber server at **192.168.1.24** to update status for the event **7PW5SXcBlxPU3jFcFJ_s** in the **aella-ser-1610496070892-** index to **Closed** and add the comment **Analysis complete.**.

Combining Updates to Tags, Status, and Comments

You can also combine updates to the Tags, Status, and Comments for an event in a single script. For example:

```
#!/usr/bin/env python

import json

import requests

headers={'Accept': 'application/json', 'Content-type': 'application/json'}

headers={'Accept': 'application/json', 'Content-type': 'application/json'}

elastic_url ='https://10.33.6.2/connect/api/update_ser'

json_data = {

"index": "aella-ser-1645142612360-",

"_id": "Jfl7SH8Boj-otBRJeTR7",

"status": "Closed",

"comments": "Analysis complete",

"event_tag": {

"op": "add",

"tag": "DevOps"

}

}

query = json.dumps(json_data)

response = requests.post(elastic_url, auth=('ohtani', '0MGDNJ5dcHPtNSWy'),
data=query, verify=False, headers = headers)

print (response.text)
```

Using the API to Verify the Update

You can always use the Stellar Cyber user interface to check whether your updates were successful – just navigate to the Event Details display for the event you changed and check the fields for your updates.

Alternatively, you can use a GET call to make sure your POST call worked. You can use any language to make the call. This ElasticSearch example uses Python with the Index and ID from the example in **Using update_ser to Update the Status and Comments Fields for Events**. The procedure is the same regardless of the type of update_ser call you made – just make sure you use the correct _index and _id in the URL.

```
headers={'Accept': 'application/json', 'Content-type': 'application/json'}

elastic_url ='https://172.16.1.31/connect/api/data/aella-ser-
1610496070892-/amsg/7PW5SXcBlxPU3jFcFJ_s'

response = requests.get(elastic_url, auth=('admin','5w3sfoGA-lz23GFR'), verify=False,
headers = headers)

print (response.text)
```

The output will be similar to:

{"_index":"aella-ser-1610496070892-
","_type":"amsg","_id":"7PW5[illegible]BlxPU3jFcFJ_s","_version":9,"_seq_no":300249,"_primary
_term":9,"found":true,"_sour[illegible]:
{"actual":1,"aella_tuples":"192.168.100.19.192.168.2.250.53.32","alert_time":16118496
00872,"appid":32,"appid_family":"Network","appid_name":"dns","appid_stdport":"yes","d
omain_list":["security.ubuntu.com"],"domain_reputation":"Good","dscp_name":"Best
Effort","dst_tuples":"00:50:56:b4:c6:ca.0.192.168.2.250","dstip":"192.168.2.250","dst
ip_assetid":"d7e9512e-c5ee-11ea-86e0-badcac5fdb29","dstip_geo":
{"city":"Unknown","countryCode":"US","countryName":"United
States","latitude":37.751,"longitude":-97.822,"region":"Unknown"},"dstip_geo_point":"
37.751,-97.822","dstip_host":"192.168.2.250","dstip_reputation":"Good","dstip_type":"
private","dstip_version":"ipv4","dstmac":"00:50:56:b4:c6:ca","dstport":53,"duration":
12,"end_reason":1,"engid":"ad56000c29af3515","engid_device_class":"Linux","engid_devi
ce_desc":"ubuntu-16.04","engid_gateway":"50.238.230.110","engid_name":"fin-vm-
2","event_category":"network","event_name":"Custom Event
2","event_score":29,"event_source":"playbook","event_type":"conn","fidelity":30,"flow
_score":98,"hostip":"192.168.100.19","hostip_assetid":"3210e0ba-c5f2-11ea-86e0-
badcac5fdb29","hostip_geo":{"city":"Unknown","countryCode":"US","countryName":"United
States","latitude":37.751,"longitude":-97.822,"region":"Unknown"},"hostip_geo_point":
"37.751,-97.822","hostip_host":"fin-vm-
2","hostip_reputation":"Good","hostip_type":"private","hostip_version":"ipv4","inbyte
s_delta":0,"inbytes_total":0,"inpkts_delta":0,"is_dga":"no","locid":"unassigned
location","metadata":{"_whitelist":-1,"is_tunneling":0,"request":
{"effective_tld":"ubuntu.com","flags":256,"message_type":"QUERY","query":"security.ub
untu.com","query_type":"AAAA","transaction_id":44968},"response":{"answers":
[{"host_type":"AAAA","name":"security.ubuntu.com","section_type":"Answer","ttl":42},
{"host_type":"AAAA","name":"security.ubuntu.com","section_type":"Answer","ttl":42},
{"host_type":"AAAA","name":"security.ubuntu.com","section_type":"Answer","ttl":42},
{"host_type":"AAAA","name":"security.ubuntu.com","section_type":"Answer","ttl":42}],"
effective_tld":"ubuntu.com","flags":33152,"message_type":"RESPONSE","query":"security
.ubuntu.com","query_type":"AAAA","reply_code":"No Error","resolved_ips":
[],"transaction_id":44968},"transaction_id":44968},"msg_class":"interflow_traffic","m
sg_origin":
{"source":"sensor"},"msgtype":4,"msgtype_name":"replica","netid":0,"netid_name":"vlan
0","obsid":3232261139,"outbytes_delta":0,"outbytes_total":0,"outpkts_delta":0,"port_n
ame":"ethernet0","processing_time":10,"proto":17,"proto_name":"udp","receive_time":16
11849138823,"response_time":10,"severity":30,"src_tuples":"00:0c:29:af:35:15.0.192.16
8.100.19","srcip":"192.168.100.19","srcip_assetid":"3210e0ba-c5f2-11ea-86e0-
badcac5fdb29","srcip_geo":{"city":"Unknown","countryCode":"US","countryName":"United
States","latitude":37.751,"longitude":-97.822,"region":"Unknown"},"srcip_geo_point":"
37.751,-97.822","srcip_host":"fin-vm-
2","srcip_reputation":"Good","srcip_type":"private","srcip_version":"ipv4","srcmac":"
00:0c:29:af:35:15","srcport":34471,"state":"Expired","tenant_name":"","tenantid":"","
threat_score":0,"timestamp":1611849134669,"timestamp_utc":"2021-01-
28T15:52:14.669Z","tos":0,"totalbytes":0,"totalpackets":0,"vlan":0,"write_by":"sef","
write_time":1611849602279,"user_action":{"last_user":"admin","history":

[{"action_user":"admin","action_time":1611881922809,"action":"Status changed to In Progress"},{"action_user":"admin","action_time":1611882046643,"action":"A comment was added by admin"},{"action_user":"admin","action_time":1611883694877,"action":"Status changed to Closed"},{"action_user":"admin","action_time":1611883805495,"action":"A comment was added by admin"},
{"action_user":"admin","action_time":1611884809614,"action":"A comment was added by admin"},{"action_user":"admin","action_time":1611884901240,"action":"A comment was added by admin"},{"action_user":"admin","action_time":1611897591837,"action":"A comment was added by admin"},
{"action_user":"admin","action_time":1612487157299,"action":"Status changed to Closed"},{"action_user":"admin","action_time":1612487157299,"action":"A comment was added by admin"}],"last_action":"A comment was added by admin","last_modified":1612487157299},"event_status":"Closed","comments":[{"comment_time":1611882046643,"comment_user":"admin","comment":"Add first comments"},{"comment_time":1611883805495,"comment_user":"admin","comment":"Add Second comments"},{"comment_time":1611884809614,"comment_user":"admin","comment":"Add Second comments"},{"comment_time":1611884901240,"comment_user":"admin","comment":"Add Second comments"},{"comment_time":1611897591837,"comment_user":"admin","comment":"Add comments at 01/28/2021, 21:19:51"},
{"comment_time":1612487157299,"comment_user":"admin","comment":"This is a demo comment"}]}}

© 2026 Stellar Cyber . All rights reserved.